

An Advanced Look at Digital Twins for Manufacturing

John Morris

24 July 2026

Assistant Professor | Dept. of Industrial & Systems Engineering and Engineering Management
The University of Alabama in Huntsville



**To industry, a digital twin is a
synchronized virtual representation
of some physical system**

Digital twins need some pedantism

The collage features five distinct digital twin web pages:

- VEXCEL DATA PROGRAM:** A dark-themed header with the VEXCEL logo. The main content area shows a city skyline with the text "High Resolution 3D Models for Cities" and a sub-header "Voxel provides off-the-shelf 3D models for cities and digital twins, urban planning".
- Matterport:** A white header with the Matterport logo and navigation links: "DIGITAL TWIN", "OVERVIEW", "EXAMPLES", and "WHAT". Below the header is a stylized illustration of a laptop displaying a 3D building model.
- SIEMENS:** A dark blue background featuring a Siemens logo and a 3D wireframe model of a machine. A man is shown interacting with the model. The text reads: "Outperform your competition with a Digital Twin" and "Leverage the comprehensive Digital Twin to design, simulate, and optimize products, machines, production, and entire plants in the digital world before taking action in the real world. This helps manufacturers to tackle industry's biggest challenges."
- DASSAULT SYSTEMES:** A white header with the Dassault Systèmes logo. The main content area shows a 3D model of a manufacturing plant with the text "Virtual Twin Experiences for Manufacturing Industries" and "Explore how virtual twin technology powered by AI revolutionizes product development, lifecycle management, supply chains and other key areas of manufacturing industries." A button at the bottom says "Watch how AI powers manufacturing".
- Microsoft Azure Digital Twins:** A white header with the Microsoft logo. The main content area shows a 3D model of a factory with the text "Azure Digital Twins" and "Create live, digital models of the physical world." Two buttons are present: "Try Azure for free" and "Create a pay-as-you-go account".

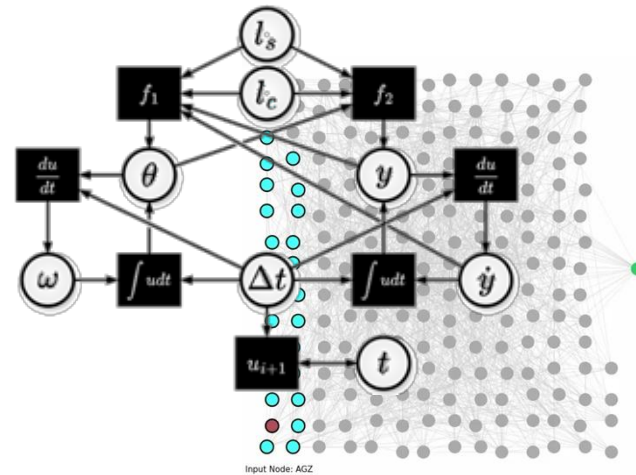
Systems theory provides the frameworks needed to define digital twins

Define digital twins from a systems perspective



Uses and limitations with current DTs

Show frameworks for next-generation DTs



Demonstration

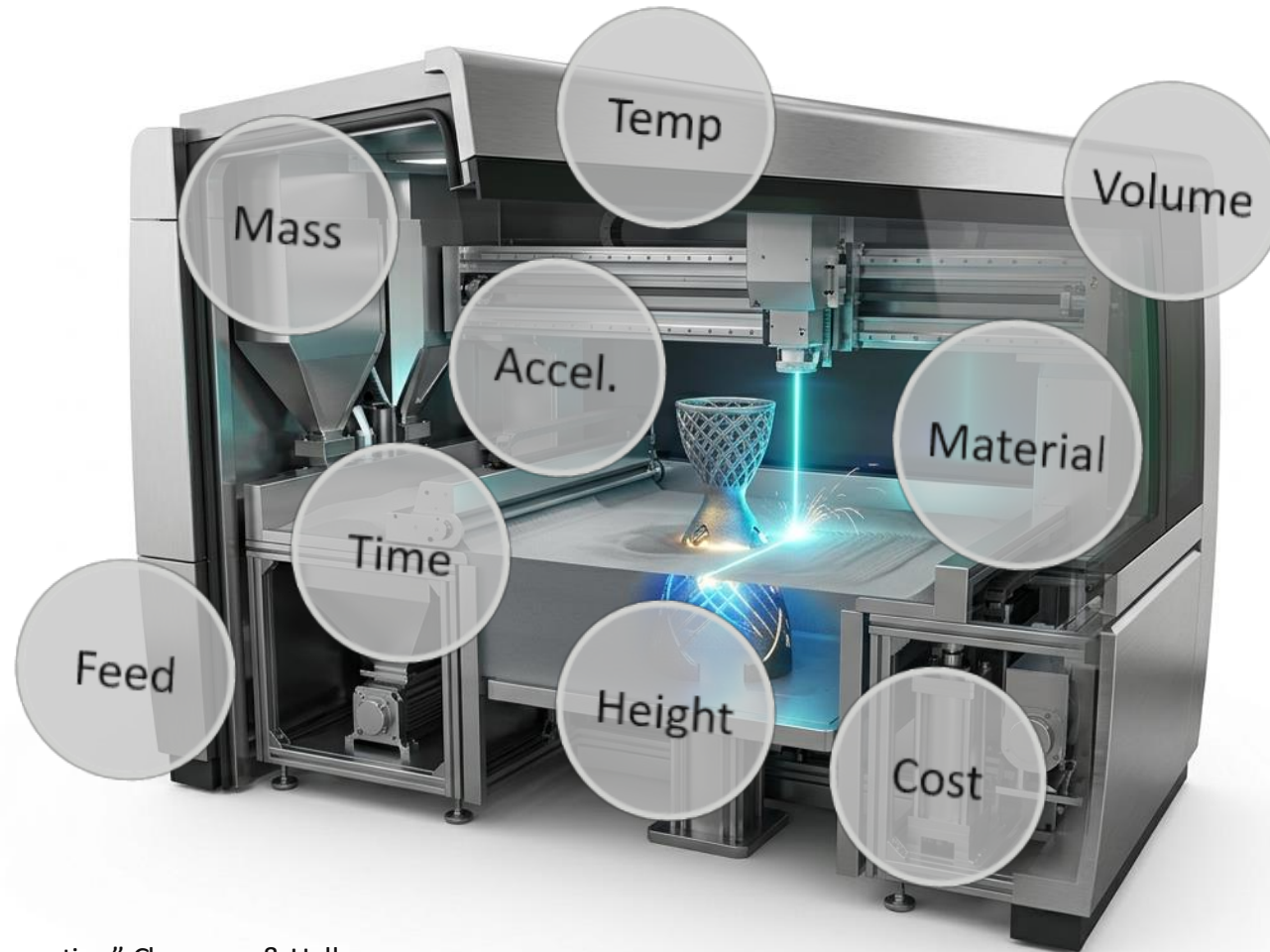
What is a Digital Twin

Digital Twins for Manufacturing

A system is something that *is*; it is *identifiable* because it behaves different than something else

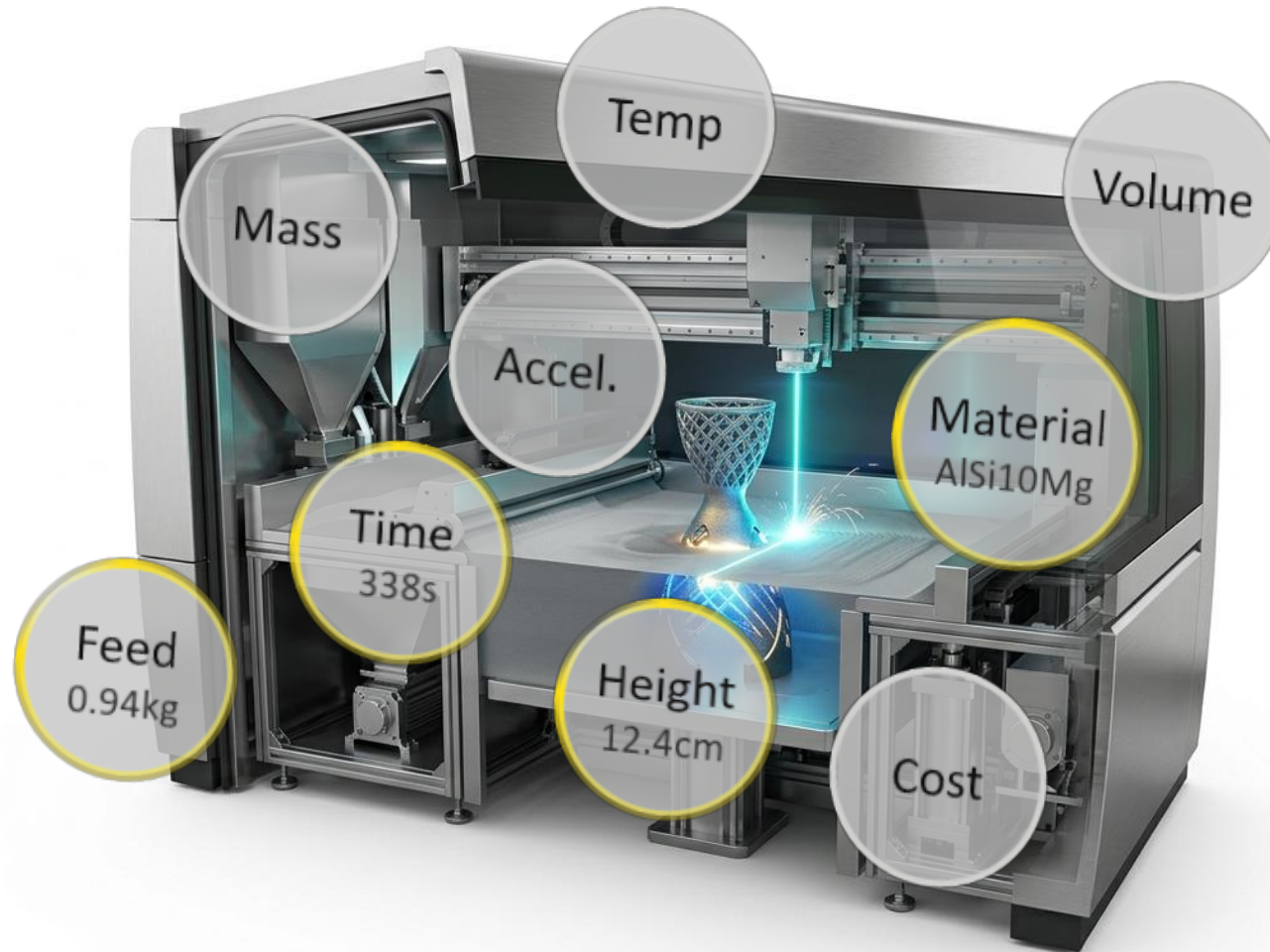


“A system is not a thing but a list of variables” – W. Ross Ashby

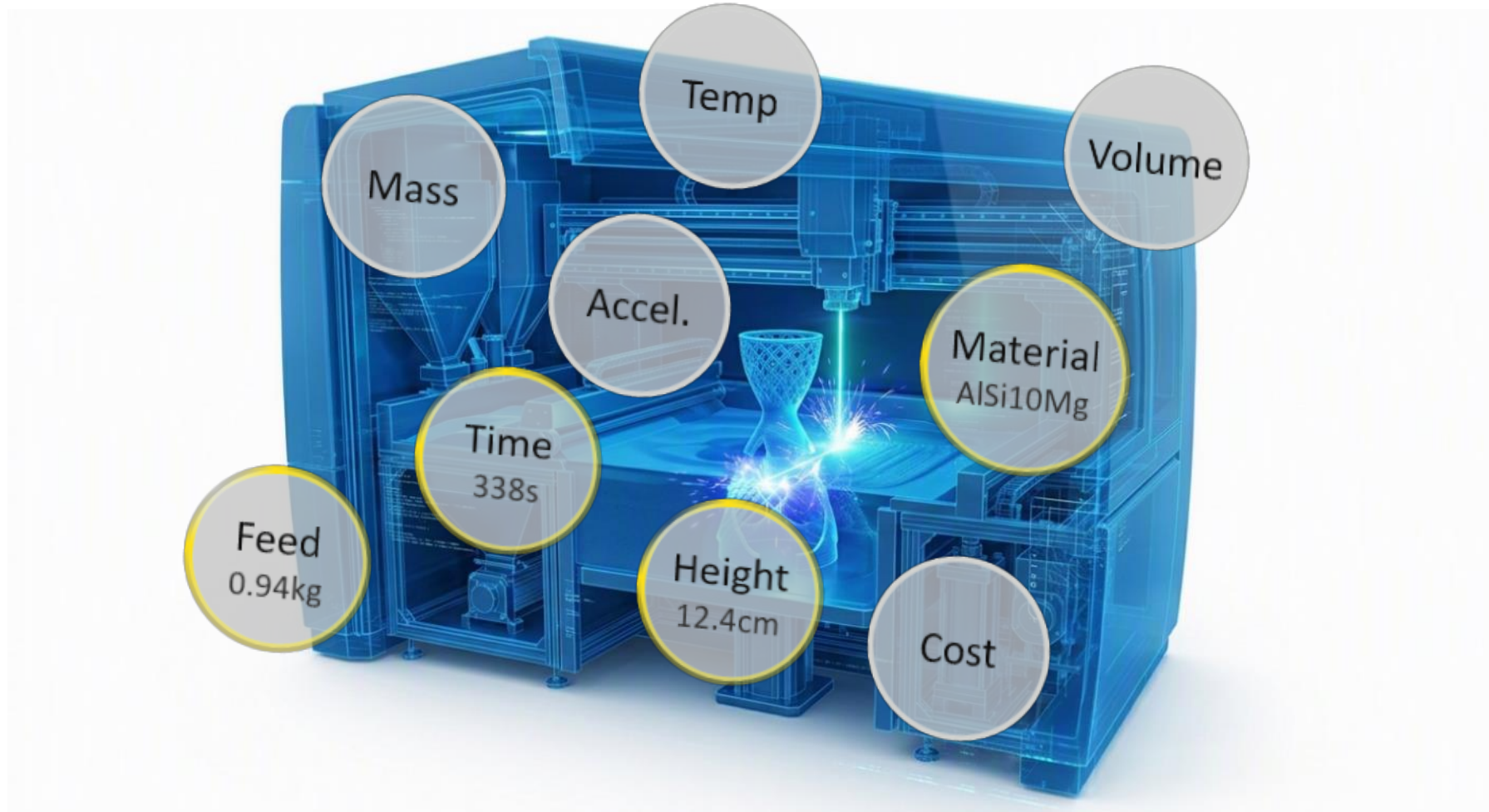


Giving a value to a variable is known as observation

Observations can be given by measurement or simulation



The value of a digital twin is providing observations for a physical system



The only role of a digital twin is to provide observations for some System of Interest (SOI)

Monitor:

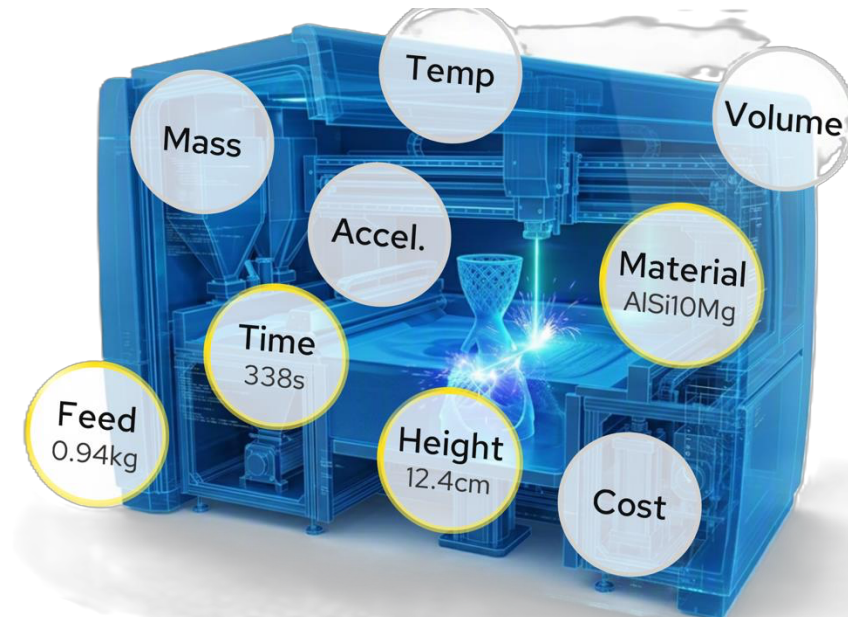
Measure SOI

Analyze:

Process measurements from SOI

Predict:

Use measurements to simulate SOI



The value of a digital twin is providing information about the SOI

How can you use information about some System of Interest?

**Predictive
Maintenance**

**Performance
Forecasting**

Fault Traversal

Role Optimization

Automation

Risk Analysis

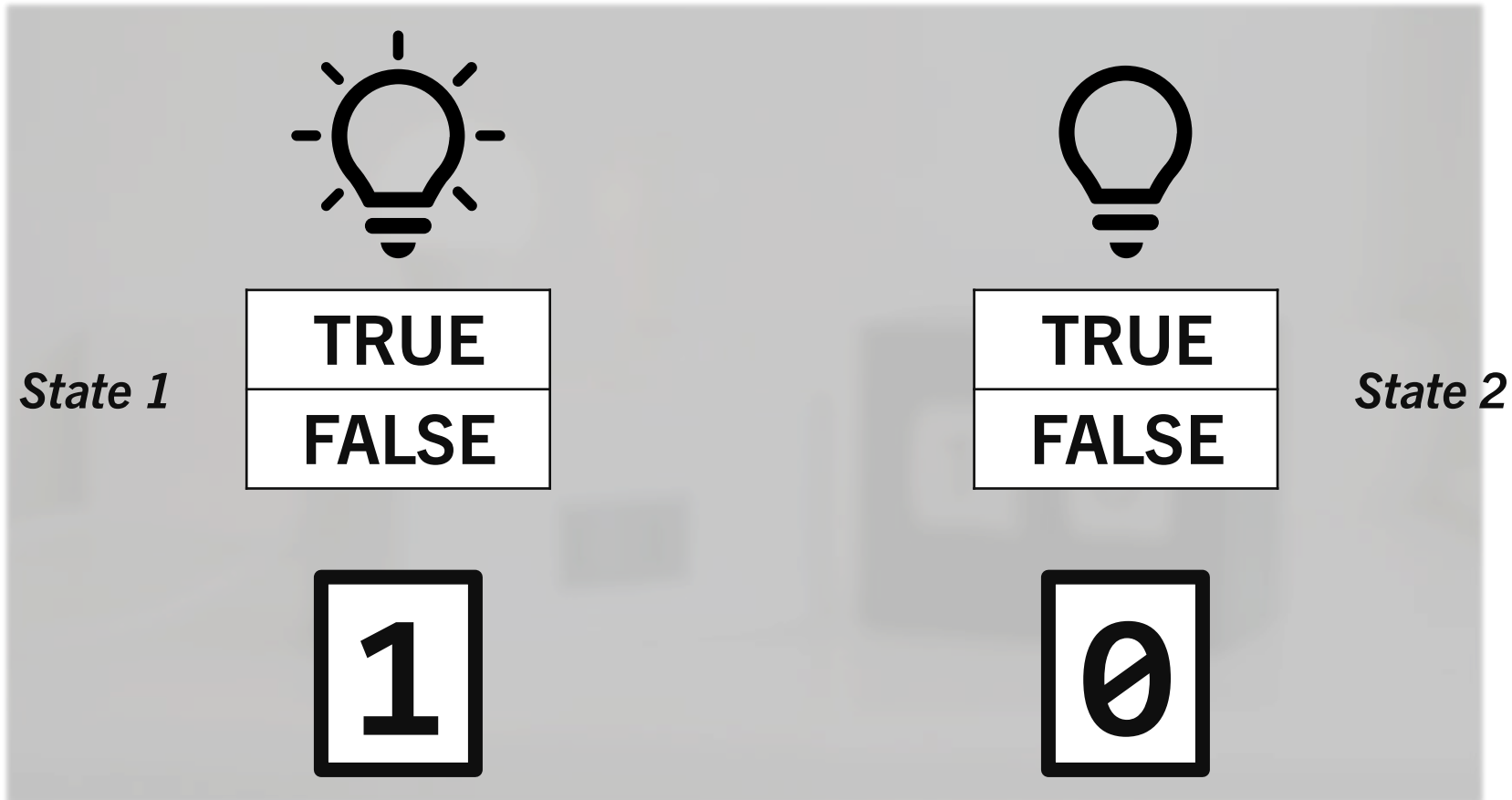
**Trade-Space
Exploration**

**Informed Decision-
Making**

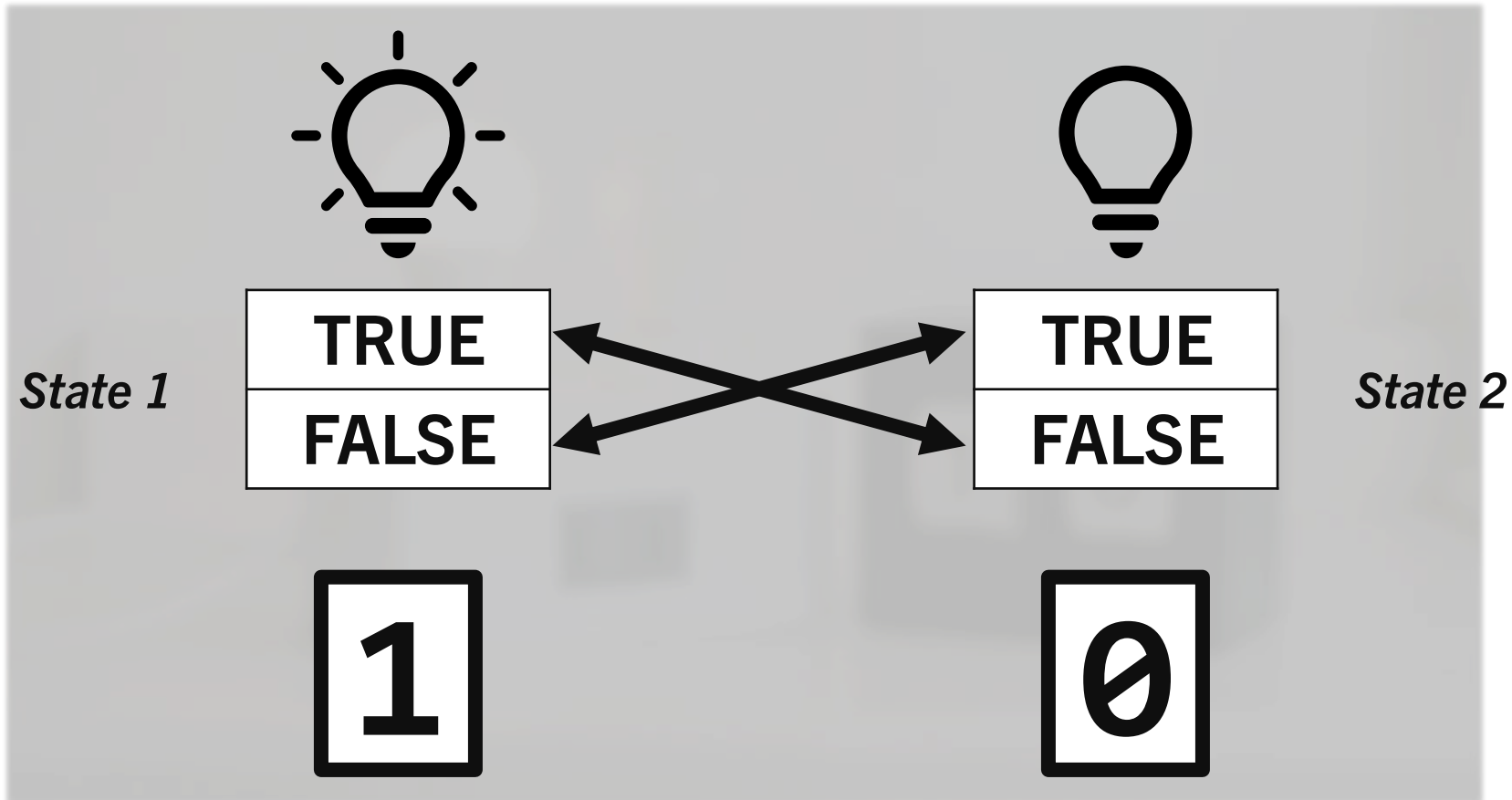
A digital twin is a system configured to enable observation of some analogous system



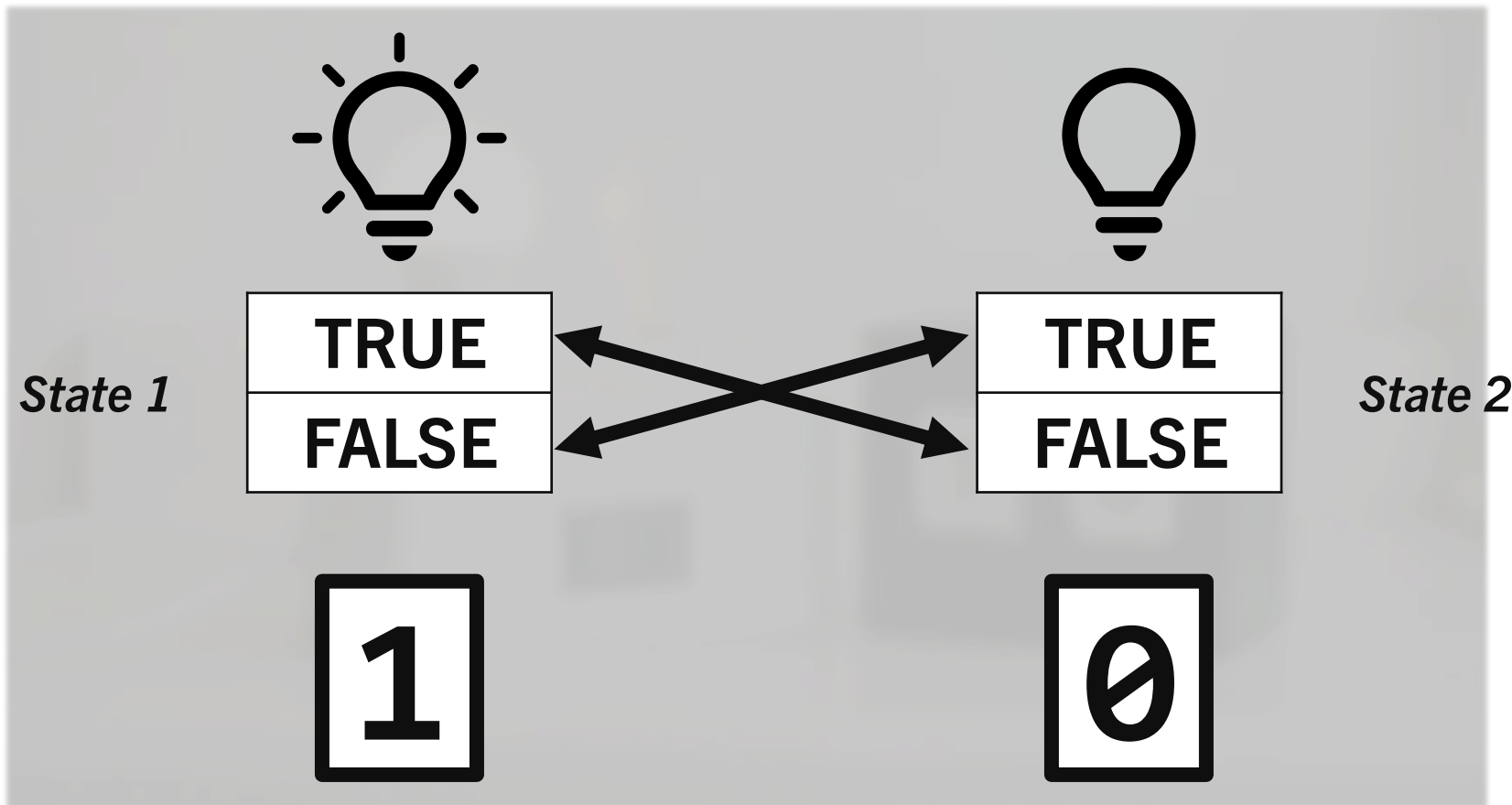
Both systems are scoped to a similar set of variables



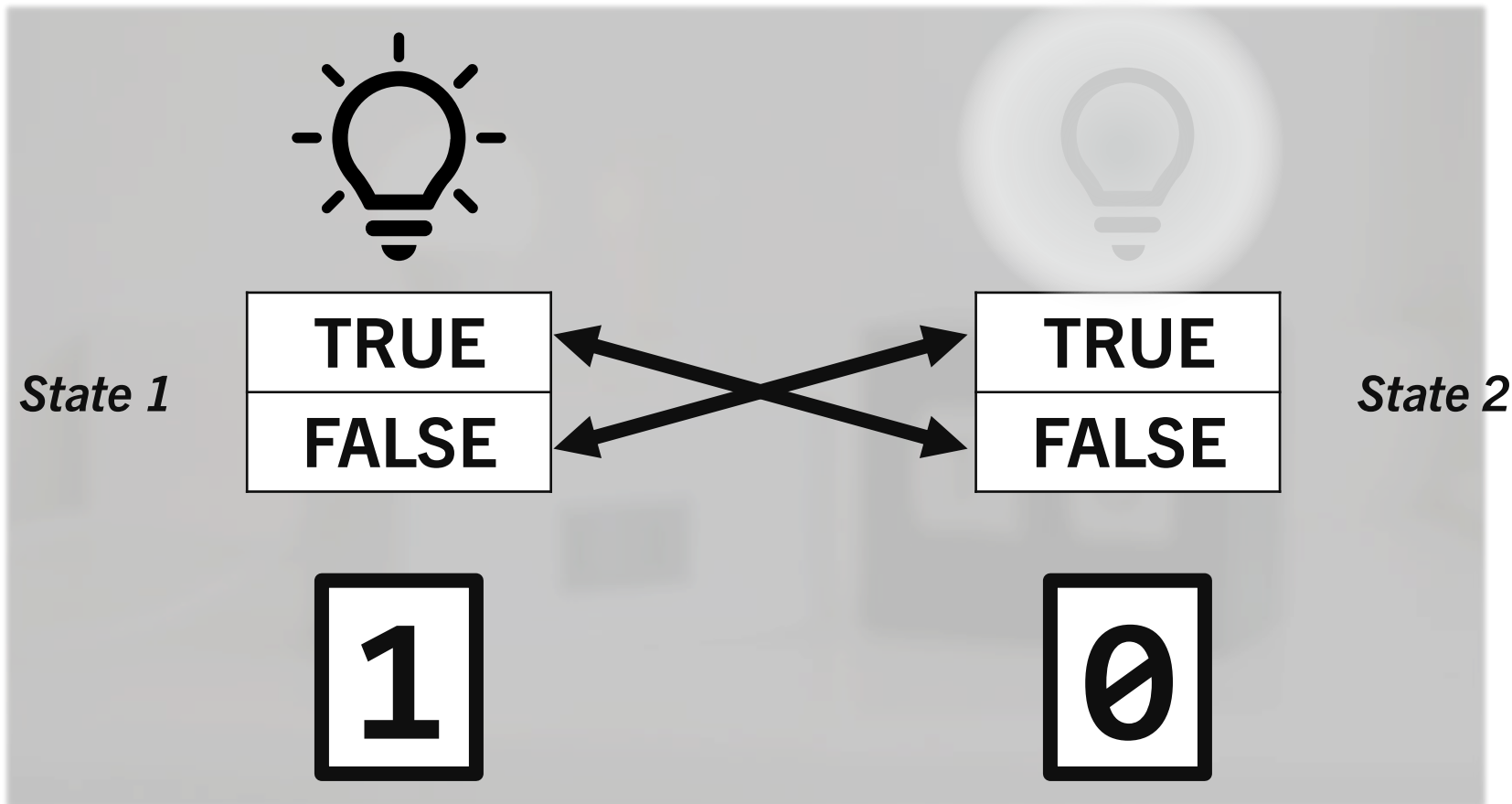
Both systems also share the same behavioral constraint



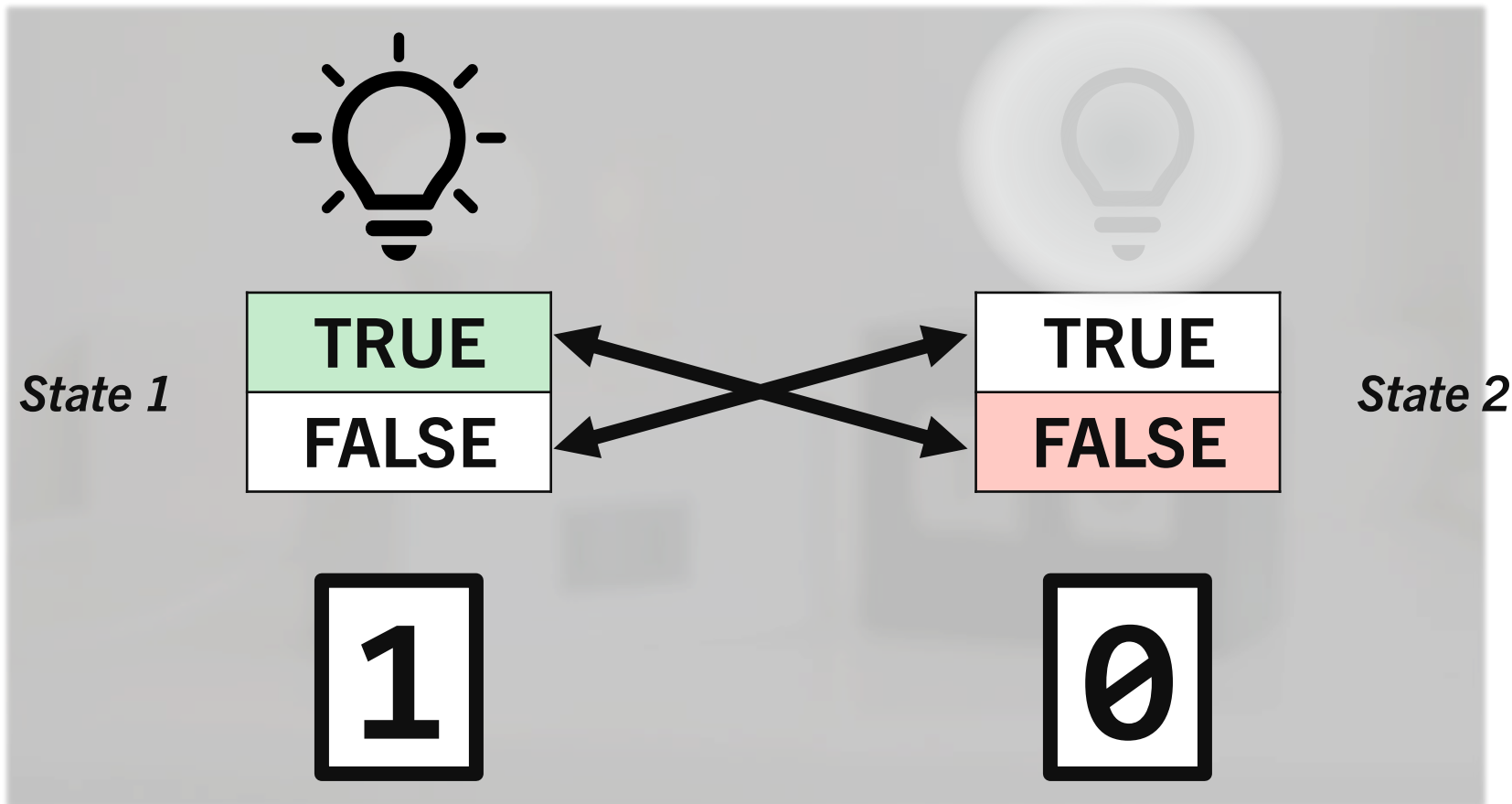
To simulate an observation requires knowledge of the system's behavior



We can simulate the SOI by measuring analogous outputs off the twin



We can simulate the SOI by measuring analogous outputs off the twin



A digital twin is a system configured to enable observation of some analogous system



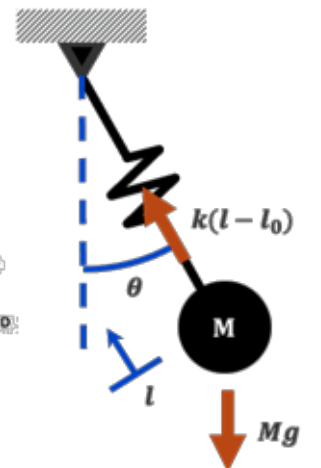
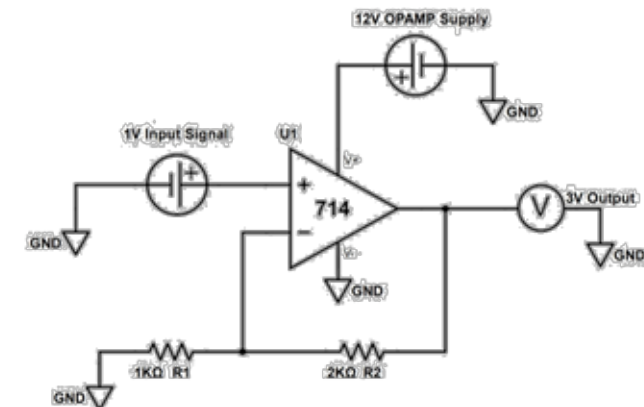
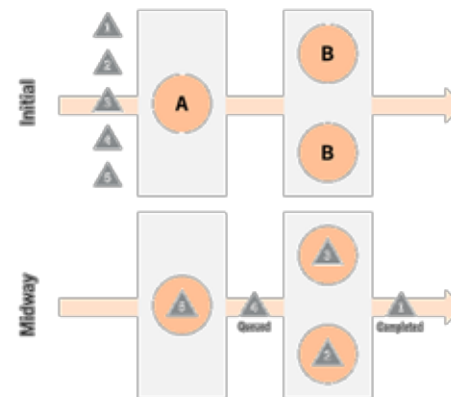
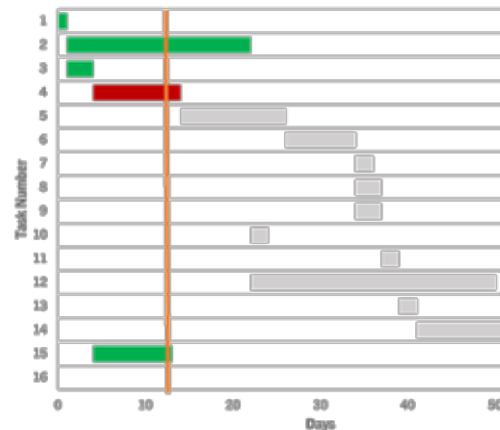
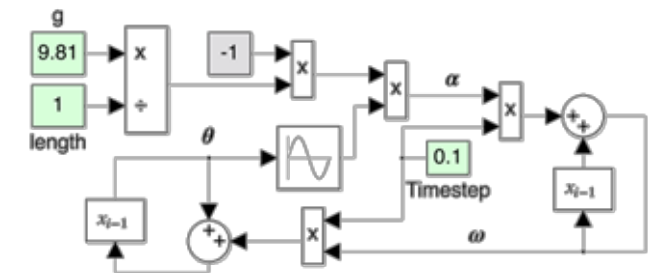
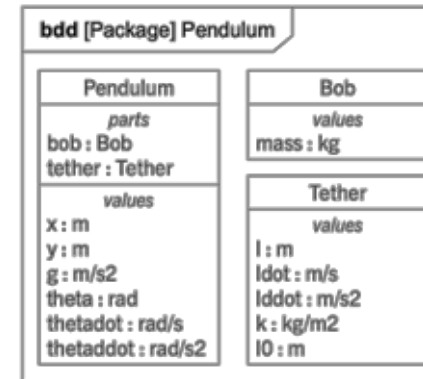
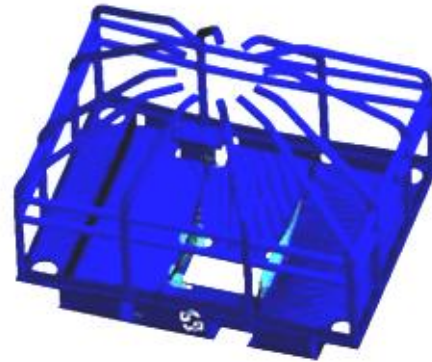
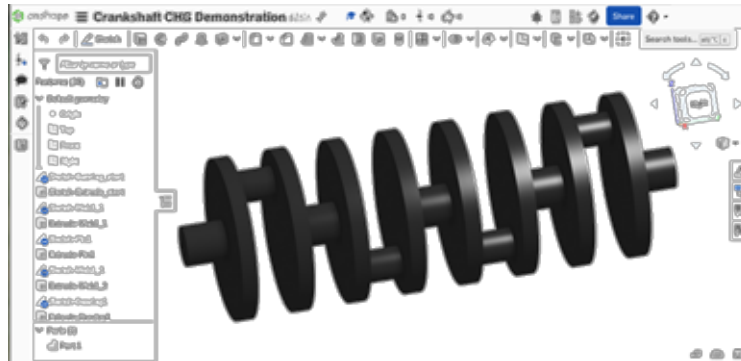
Synchronization mechanisms are not part of the digital twin



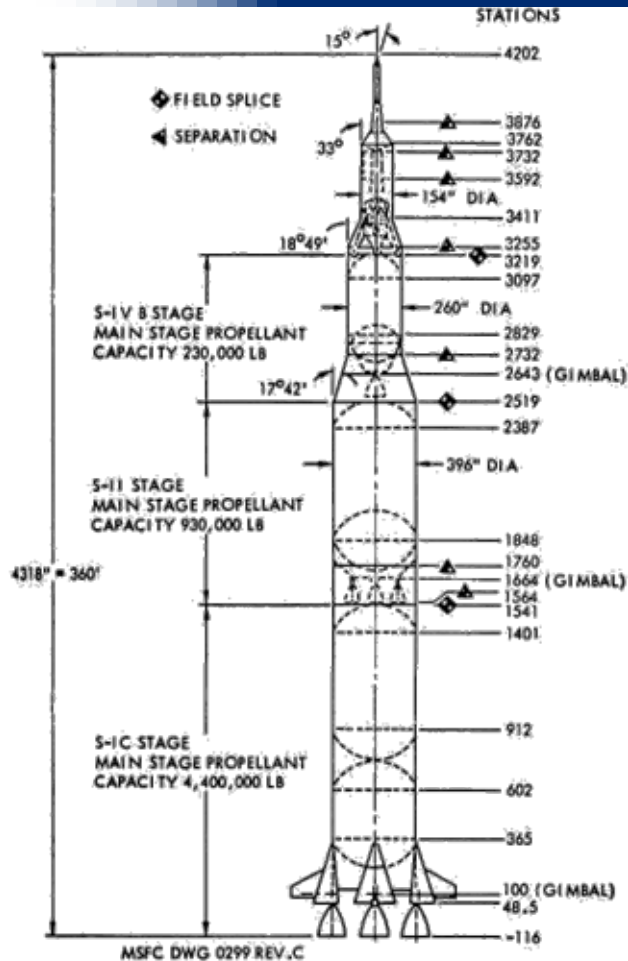


Problems with digital twins stem from maintaining meaning when connecting information for complex systems

Understanding complex systems requires integration of heterogeneous information and models



Issues with modeling—such as uncertainty, verification, or ignorance—are not unique to digital twins



Saturn V Reference Dimensions

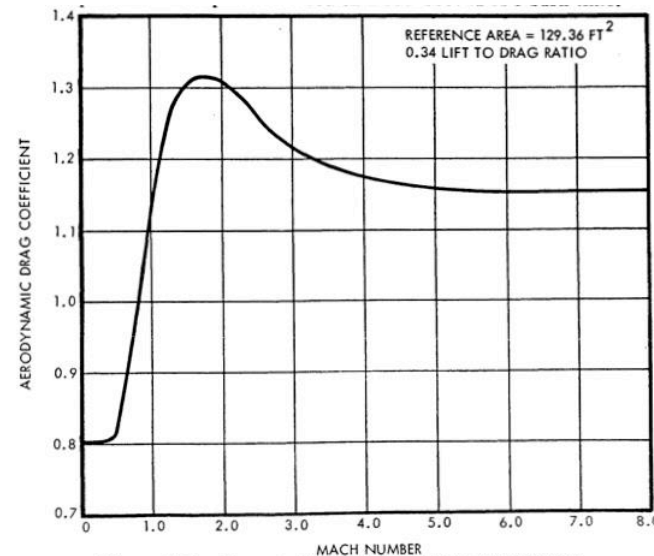
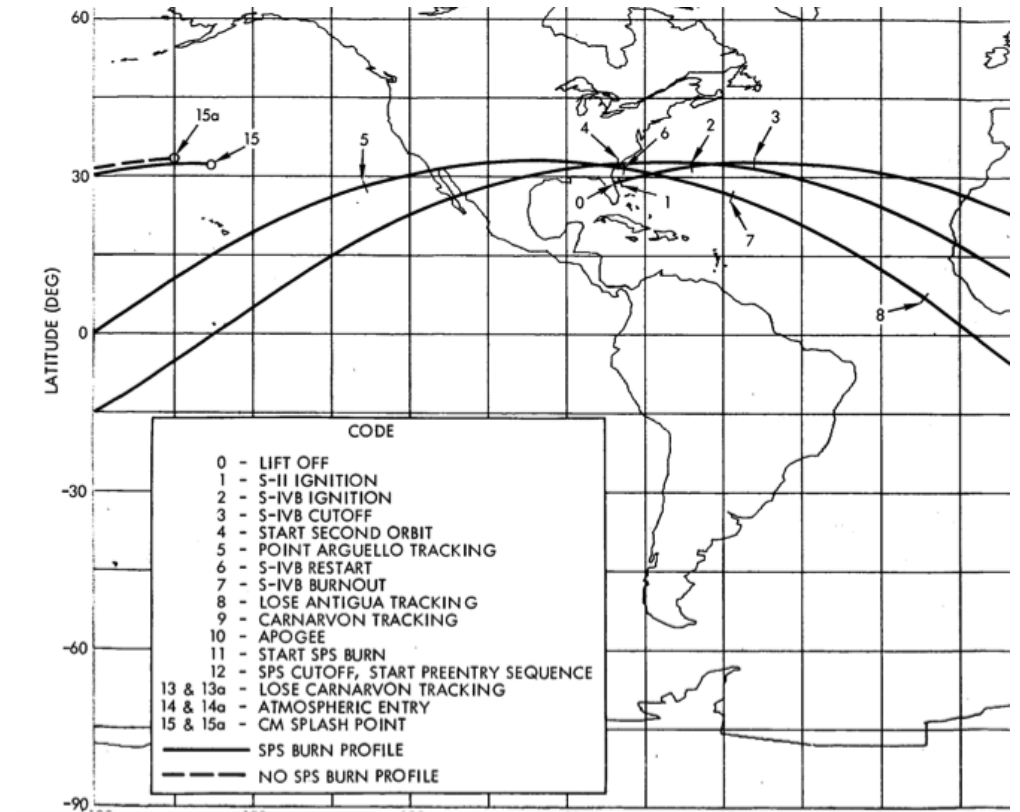
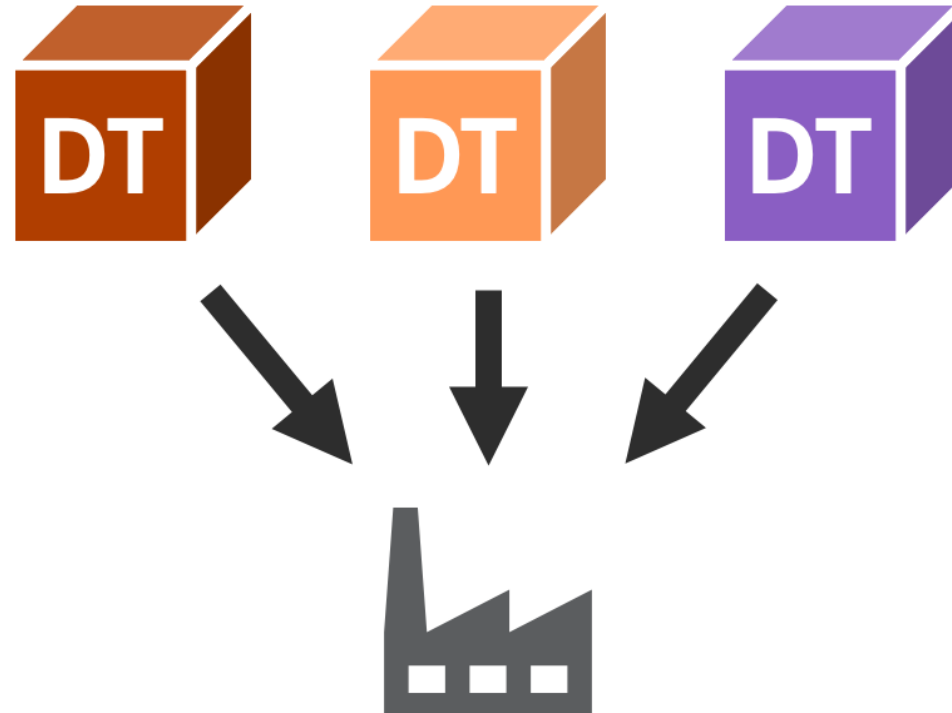


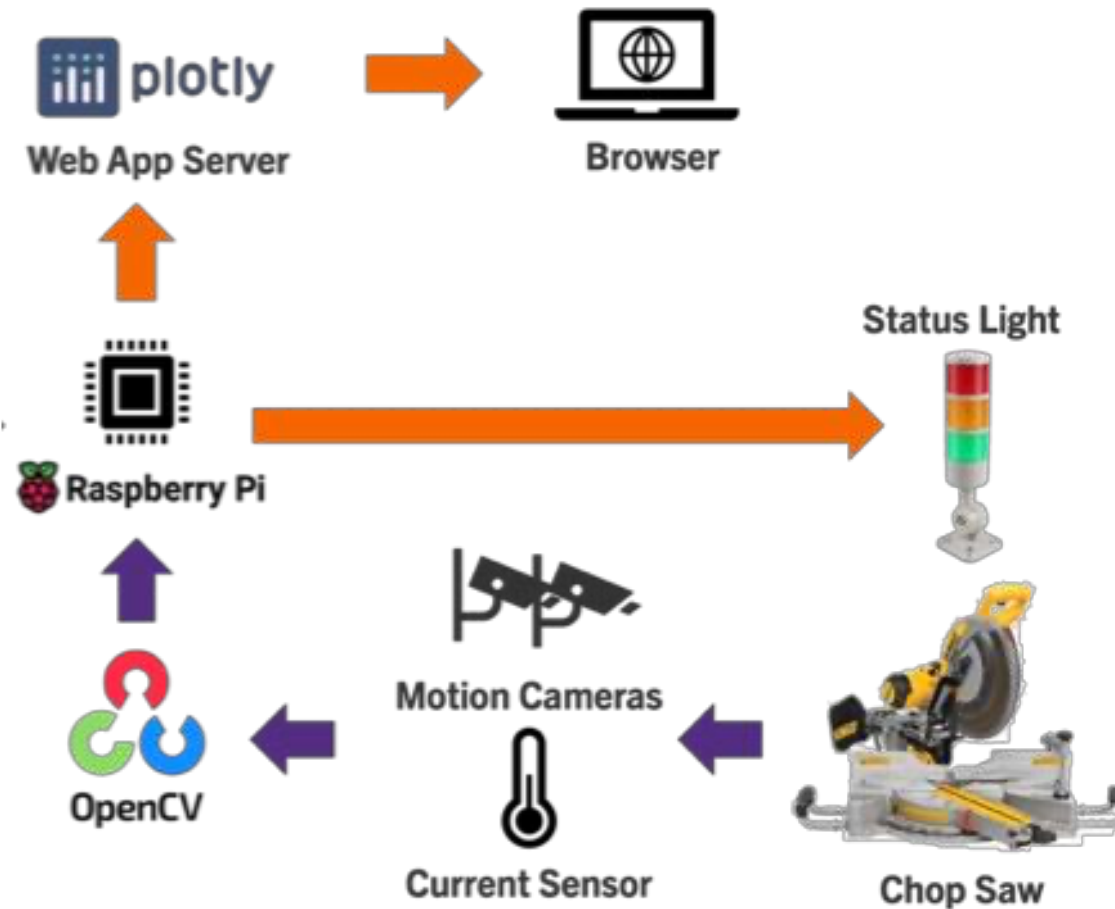
Figure 3-3. Command Module Trim Drag Coefficient



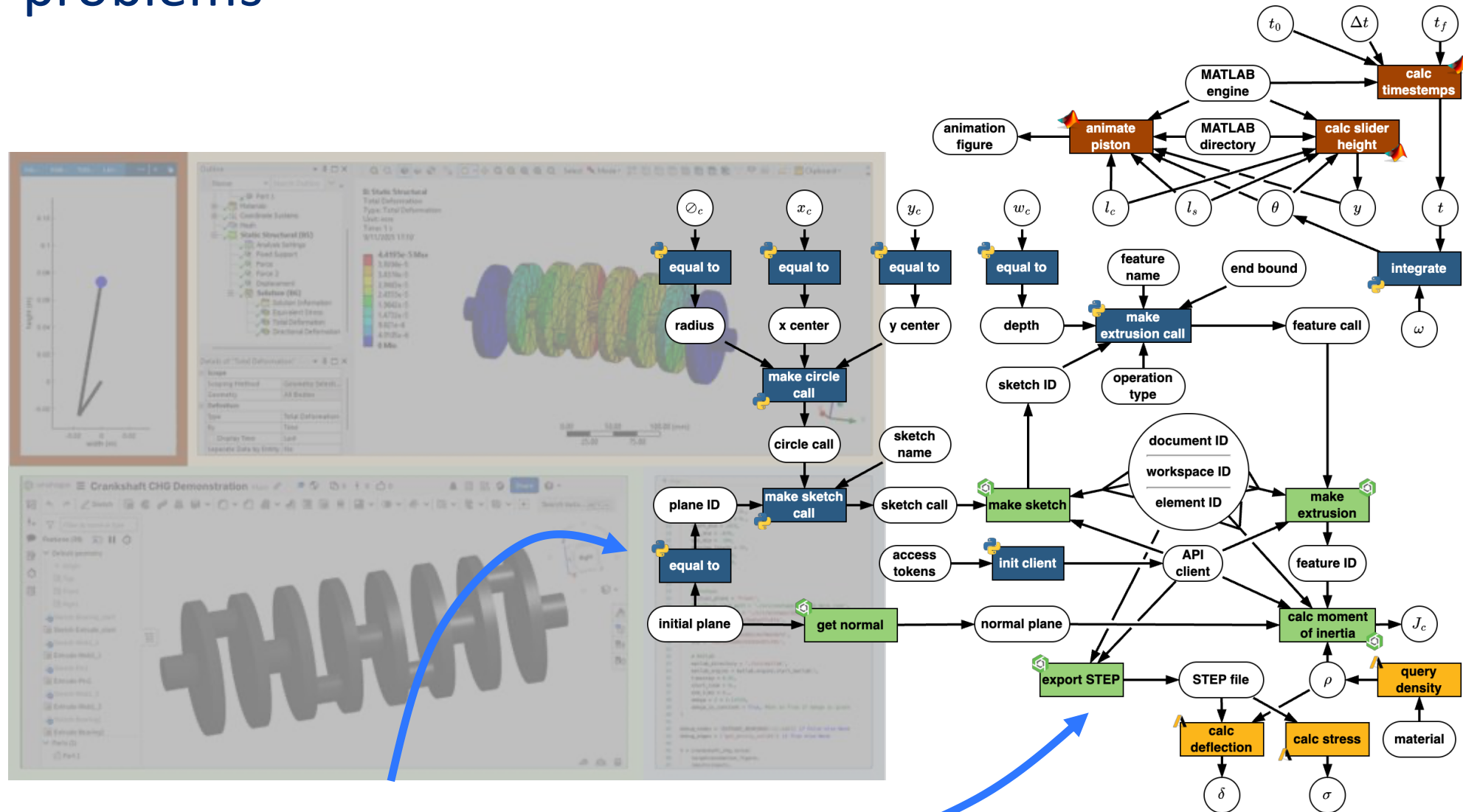
Emergent behavior arises when trying to combine subsystem digital twins



Complex behaviors require advanced software to solve, all which must be connected to make the digital twin



Using constraint hypergraphs (CHGs) mitigates many of these problems



independent software

Constraint Hypergraphs

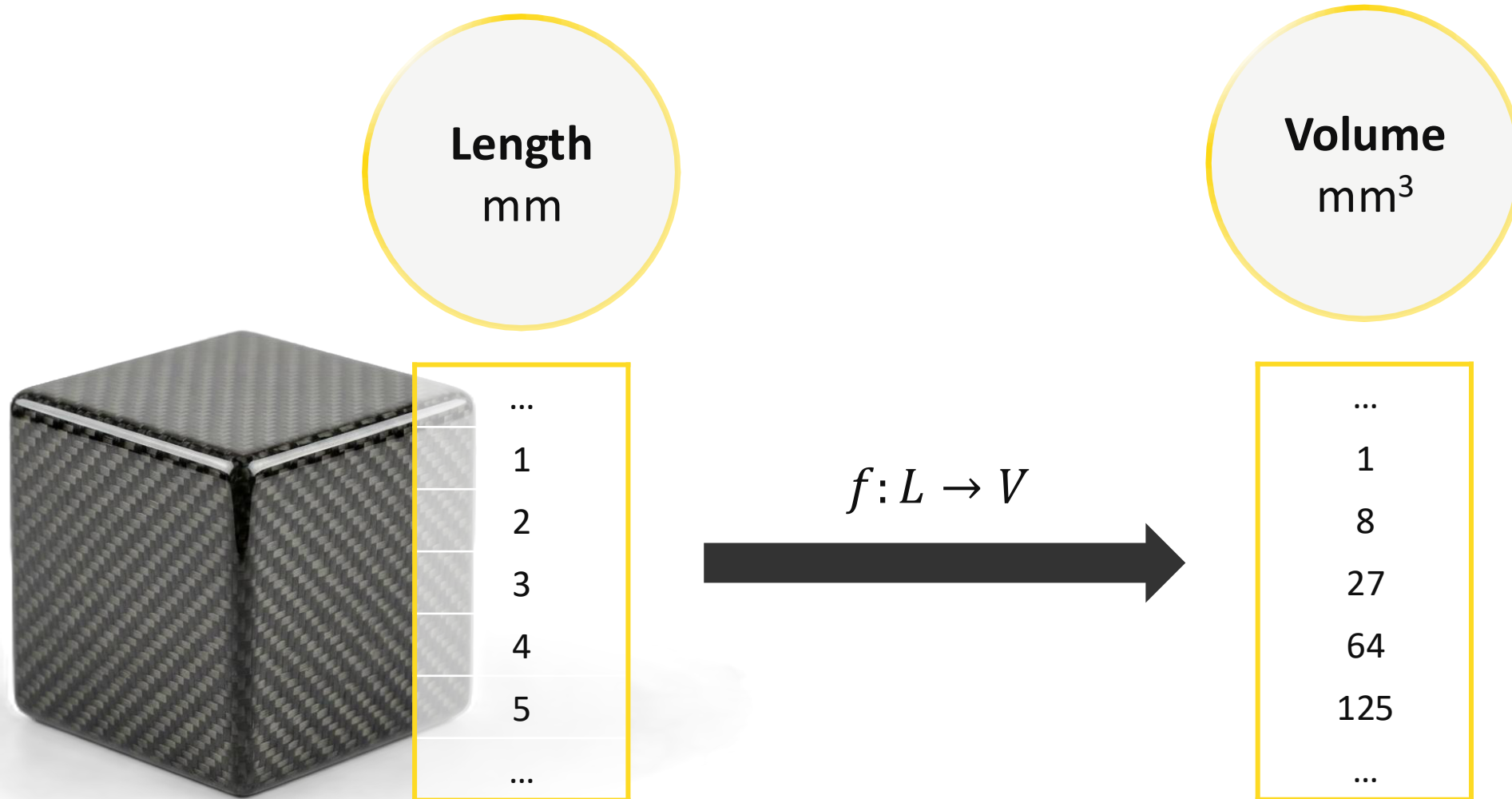
Digital Twins for Manufacturing

The behavior of a system is given by the relations, or constraints, between system variables

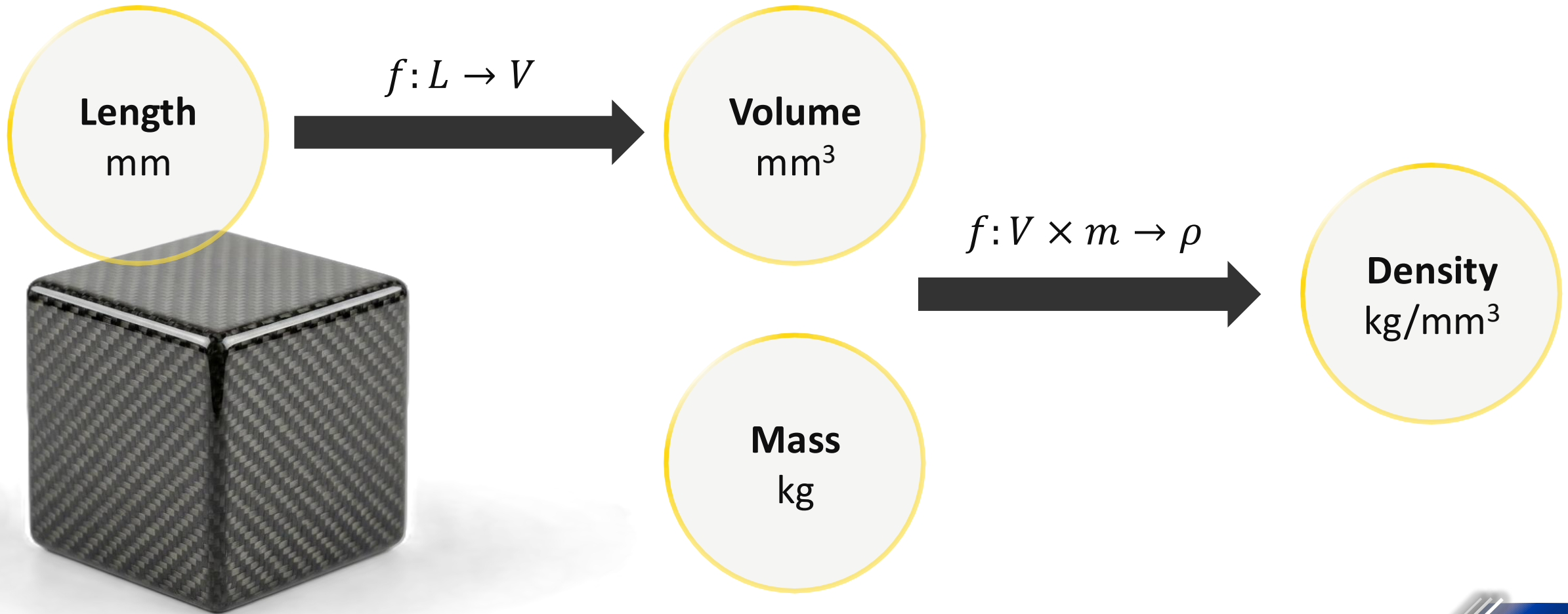
What is the behavior of this cube?



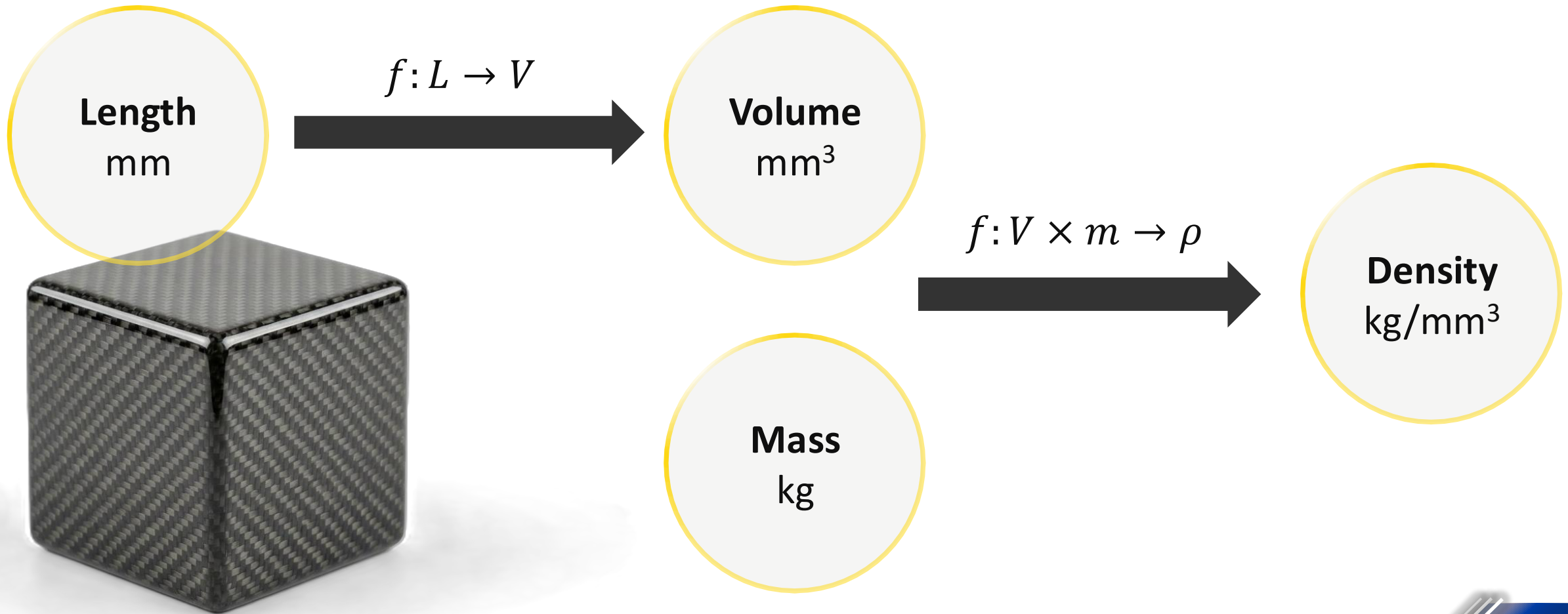
If variables are sets, then relationships can be represented as functions



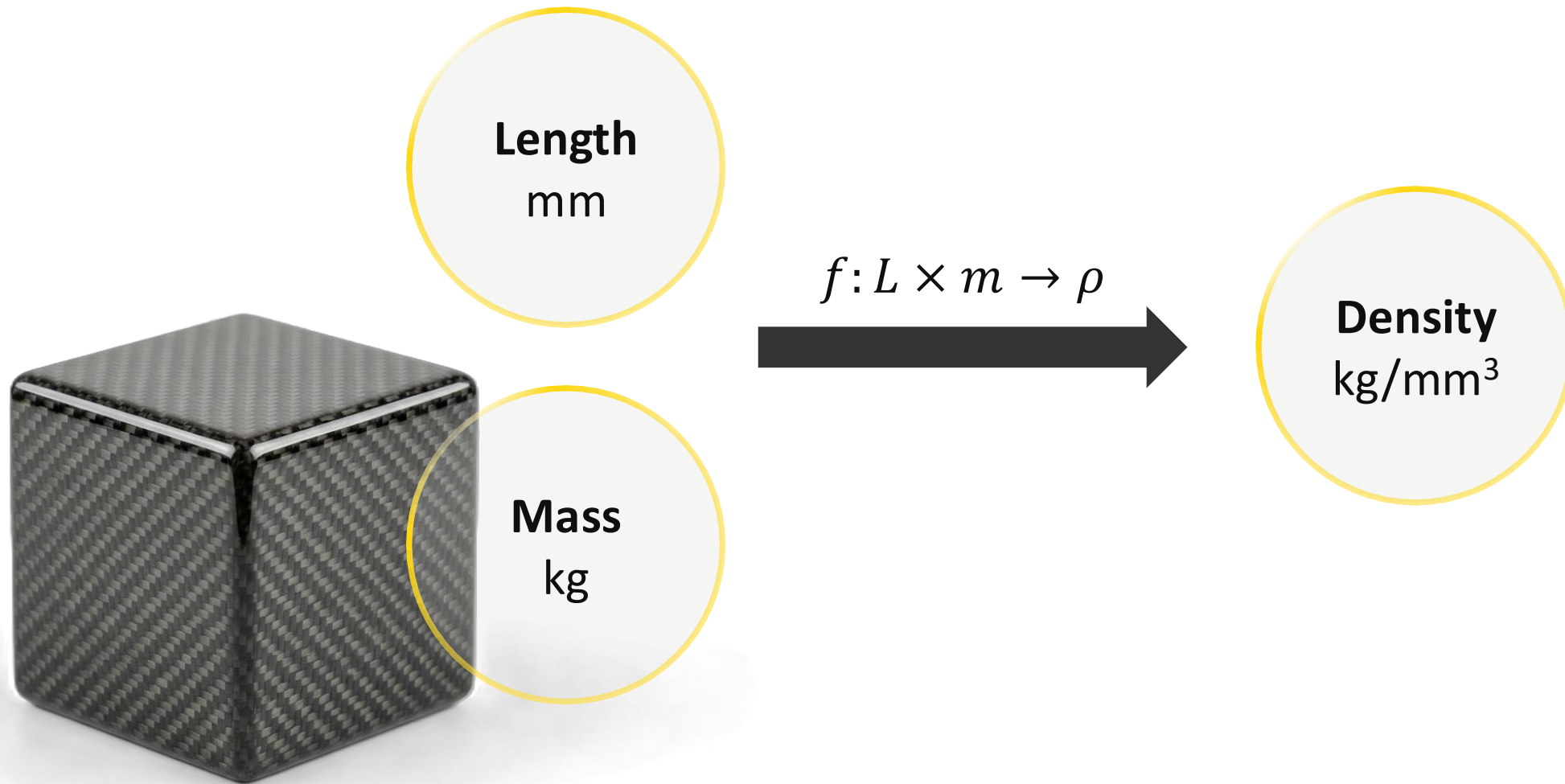
If variables are sets, then relationships can be represented as functions



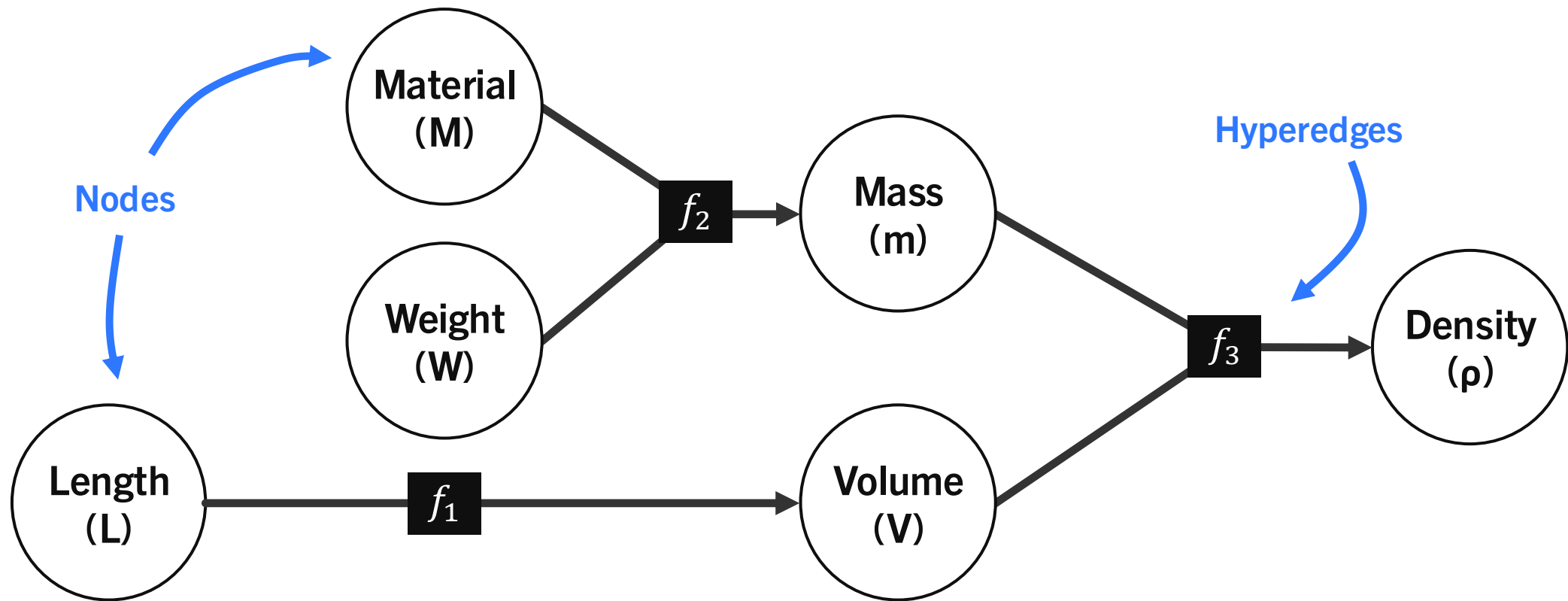
What inputs are needed to observe Density?



Functions compose to make new functions

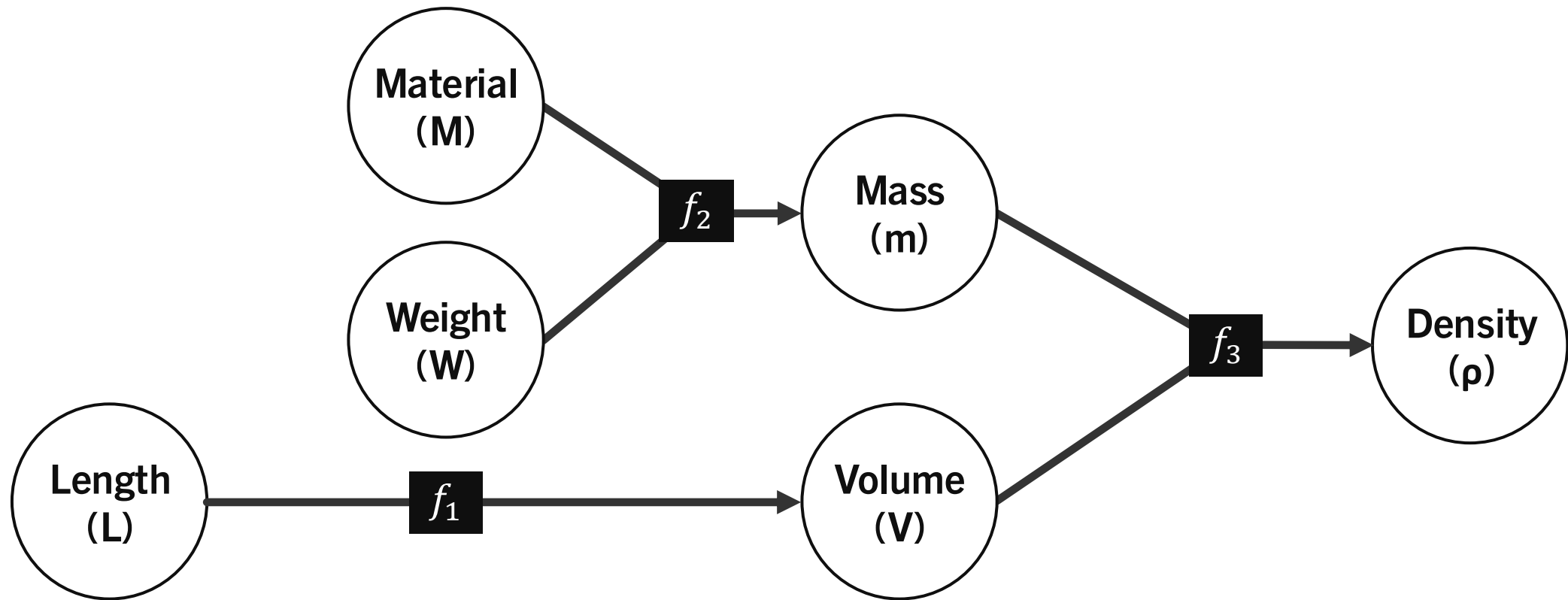


A constraint hypergraph is a mathematical structure consisting of all the variables (nodes) and behaviors (edges) modeled for a system

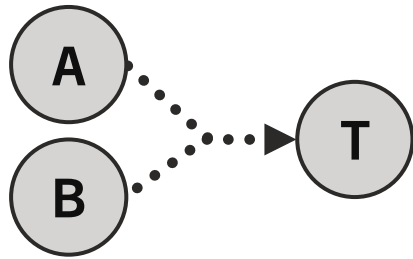


CHGs are purely functional, reducing models to their primitive constituents

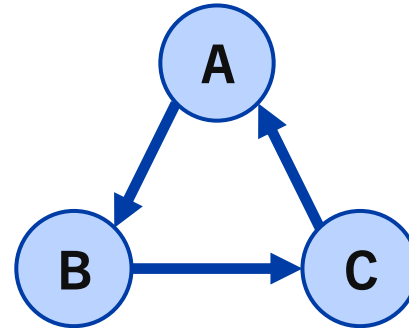
Consequently, all models can be represented as a CHG



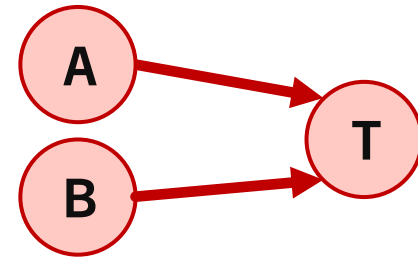
There are three structures in a CHG that complicate modeling



Partial Edges

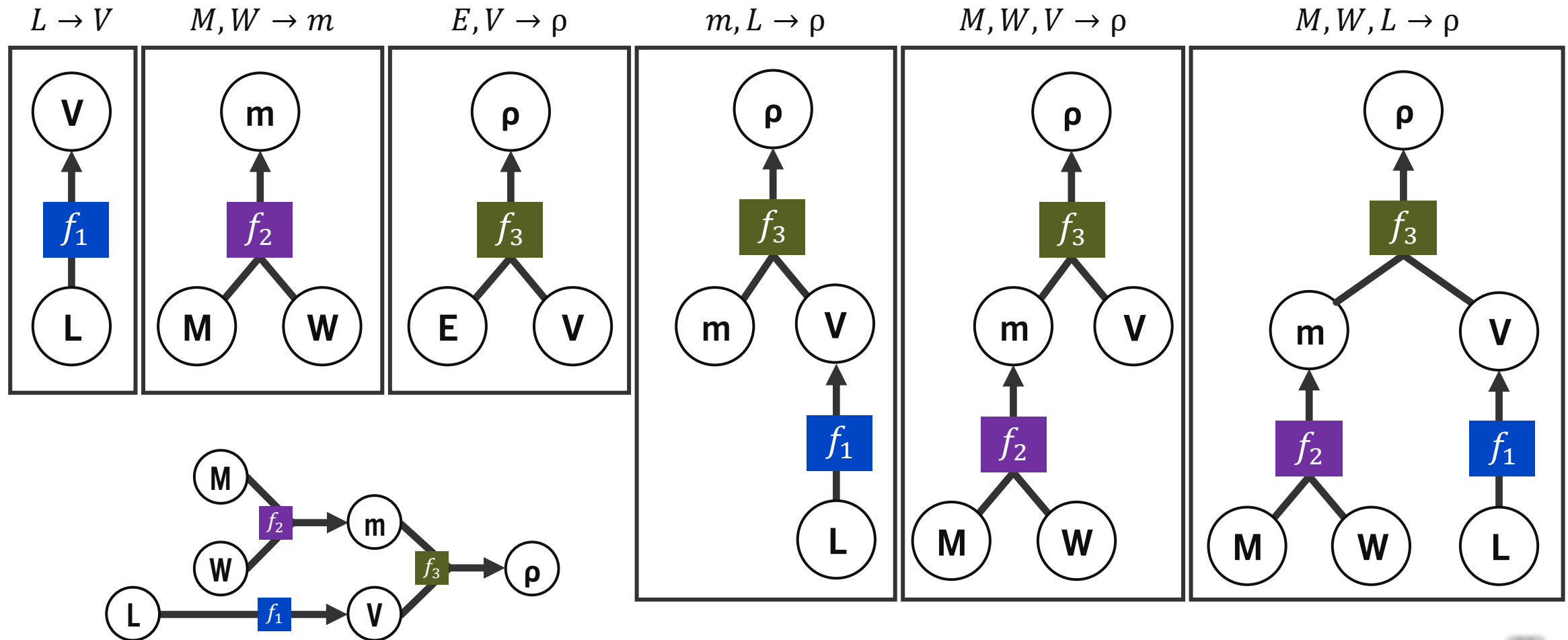


Cycles



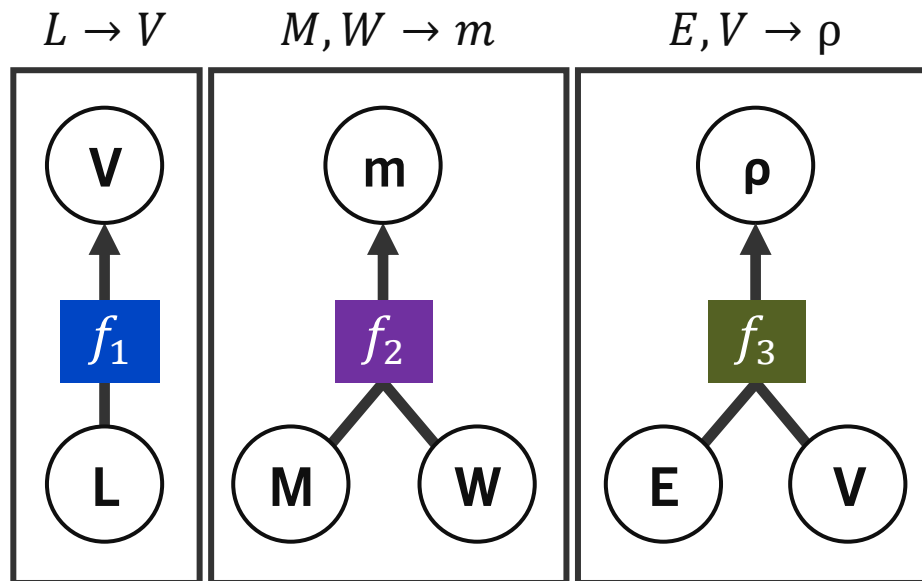
Multi-Edges

All paths through a CHG are a valid simulation

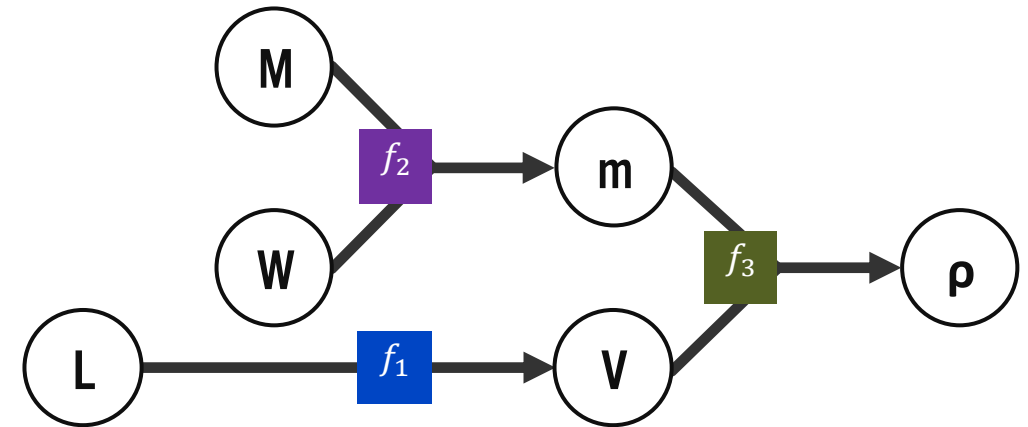


Pathfinding enables declarative simulation

CHGs are a fully simulatable framework



Imperative



Declarative



Declarative simulation provided by ConstraintHg



constrainthg 0.3.2

```
pip install constrainthg
```

Kernel for building and simulating constraint hypergraphs.

Navigation

- Project description
- Release history
- Download files

Project description



JOSS 10.21105/joss.09131 docs passing test/linter passing release v0.3.2 last commit february



Search

SOFTWARE

- Quickstart
- Tutorial
- API
 - Node
 - Edge
 - Hypergraph
 - Relations Module
- About
- Demos
- Repository

CONSTRAINT HYPERGRAPHS

- CHG Overview
- Learn About CHGs

Quickstart

Use [PIP](#) to install ConstraintHg into your Python environment:

```
pip install constrainthg
```

From there you'll want to import the library into your Python script. This is a pretty typical method to use:

```
from constrainthg.hypergraph import Node, Hypergraph
import constrainthg.relations as R
```

Simple Demo

Note that this demo is found in [demos/demo_basic.py](#)

Let's build a basic constraint hypergraph of the following equations:

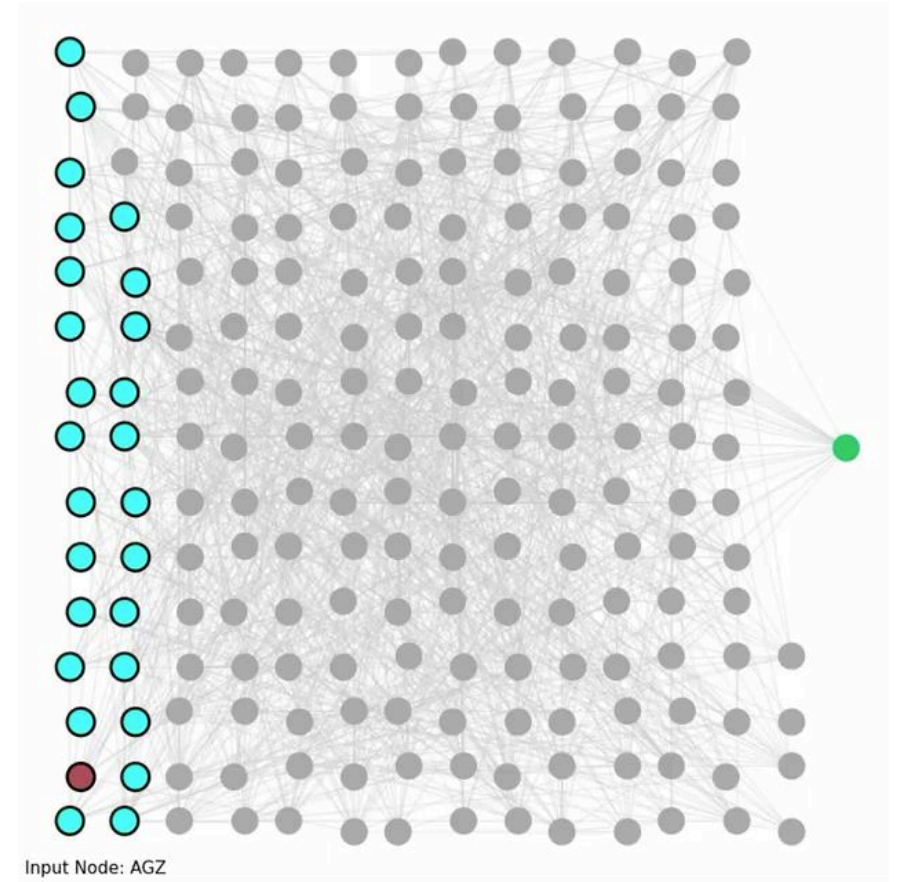
- $A + B = C$
- $A = -D$
- $B = -E$
- $D + E = F$
- $F = -C$

latest



ConstraintHg Functionality

- Provides methods for:
- Representing and visualizing CHGs
- Forming cycles, partial edges, and edge weights
- Pathfinding for declarative simulation
 - Extensive logging tools
- Merging CHGs



Process of making a CHG

1. Identify system facts (nodes)

Parameters, application variables, API tokens, etc.



2. Form relations (edges) between nodes

Relations show how one node is determined by a set of other nodes



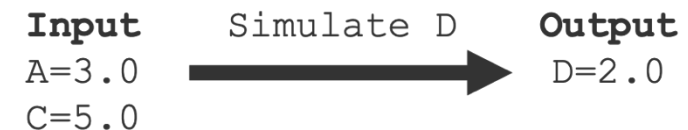
3. Pass to CHG solver

Solver parses the CHG

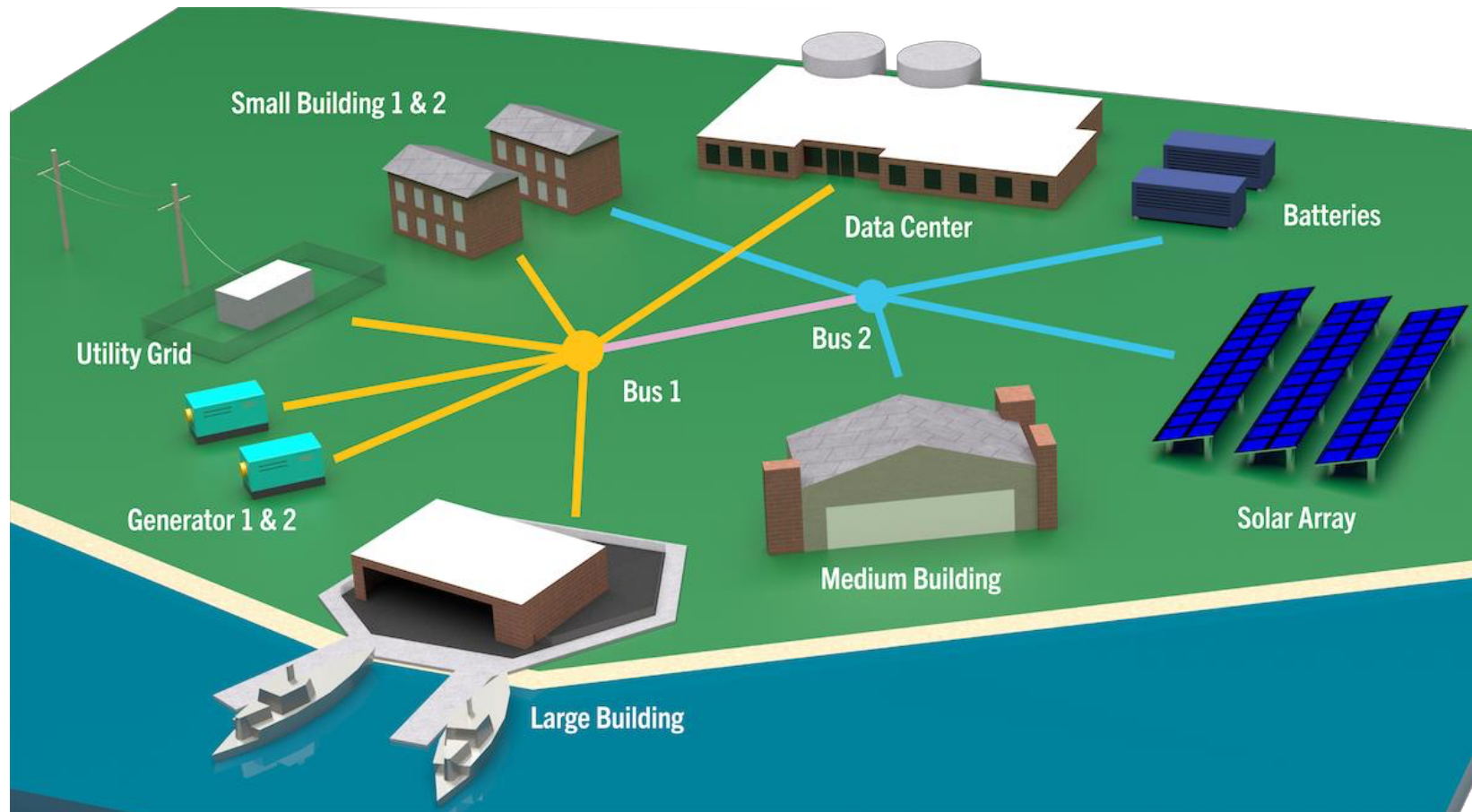


4. Request simulation

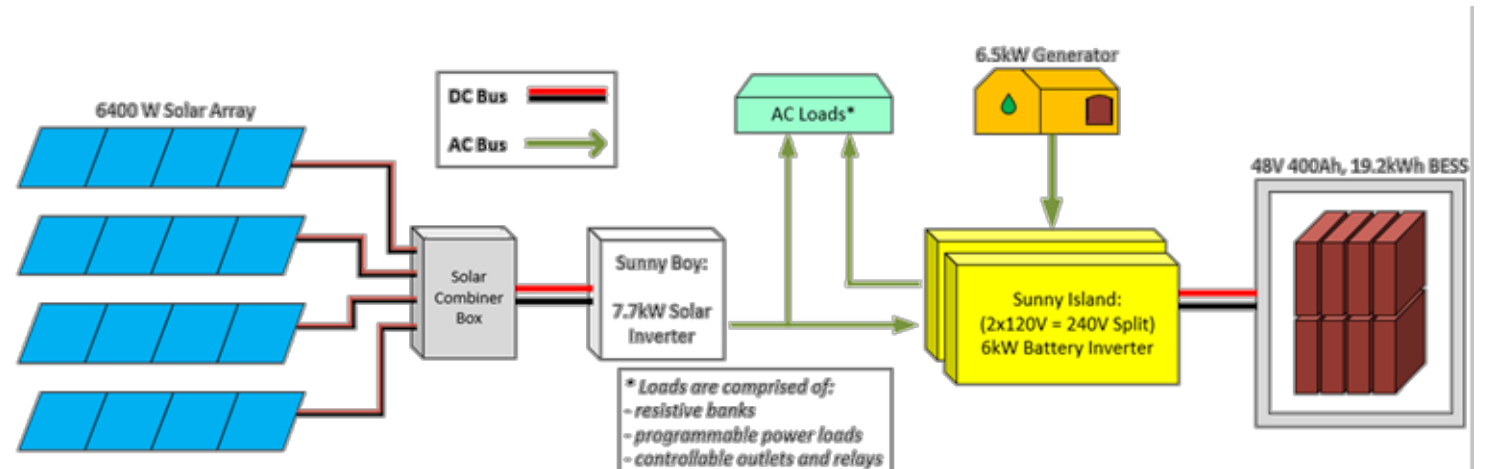
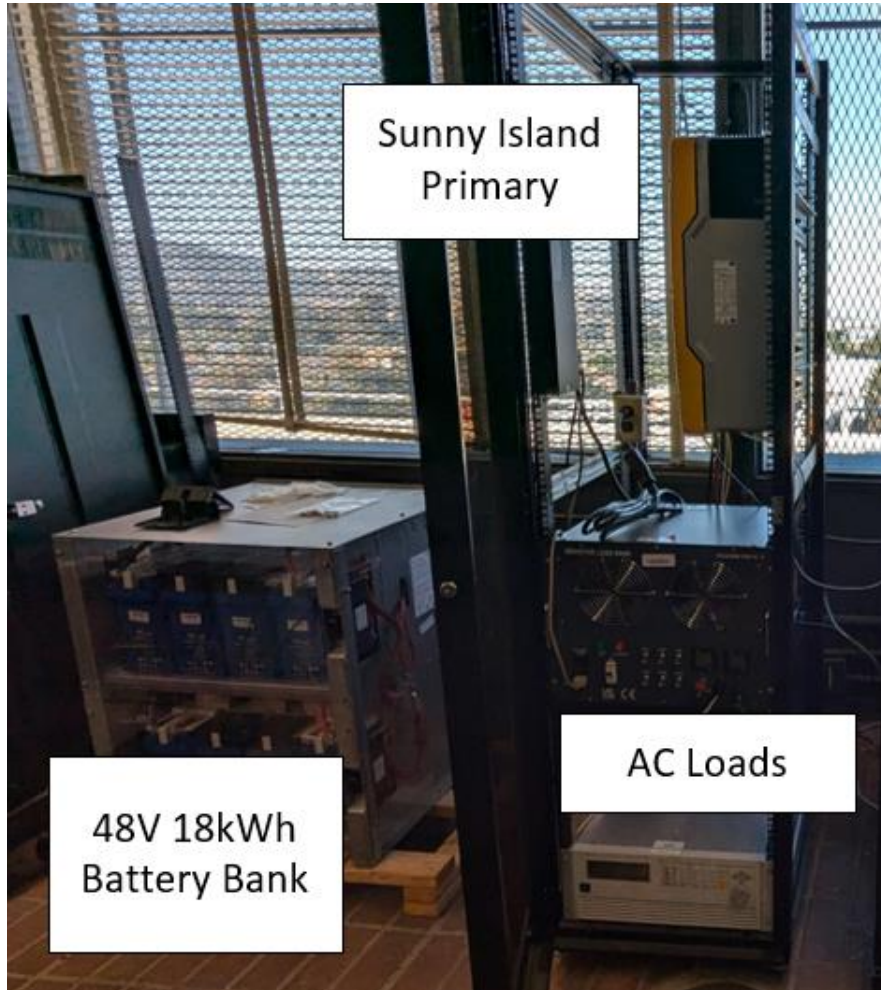
Solver simulates requested output by finding the shortest path mapping it to a set a known inputs



Demonstration: Microgrid Digital Twin



Physical Microgrid at Naval Postgraduate School

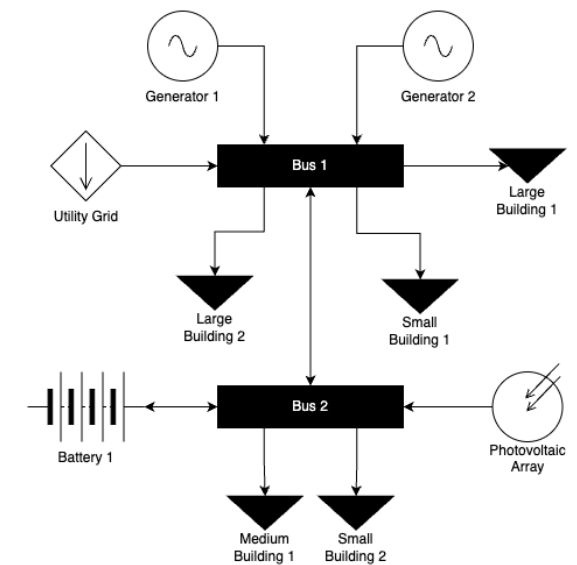
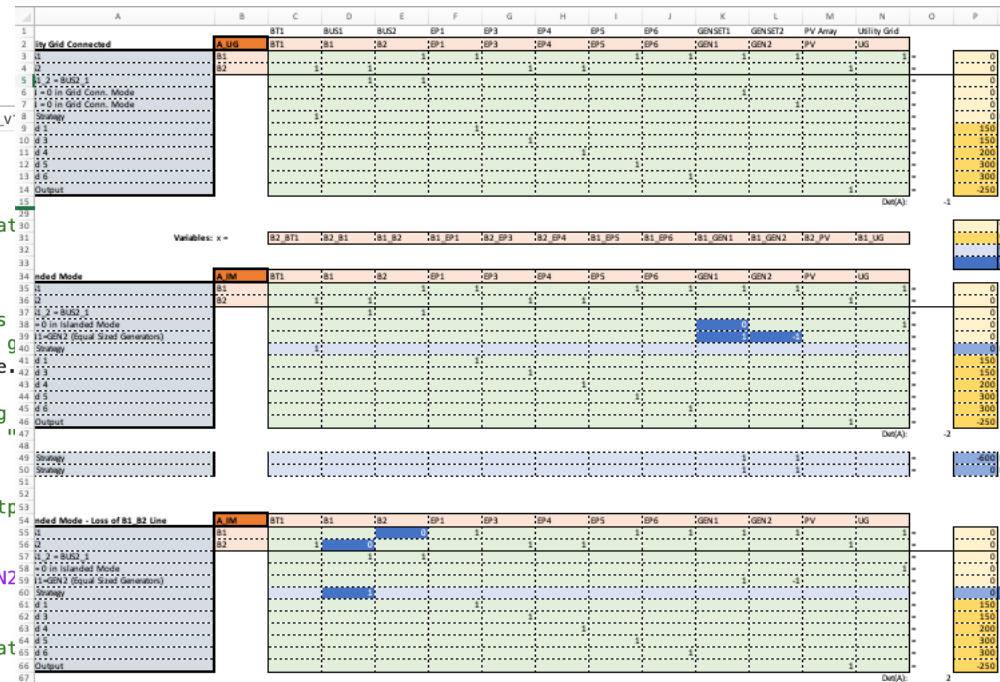


Previous Imperative Modeling of Microgrid

```

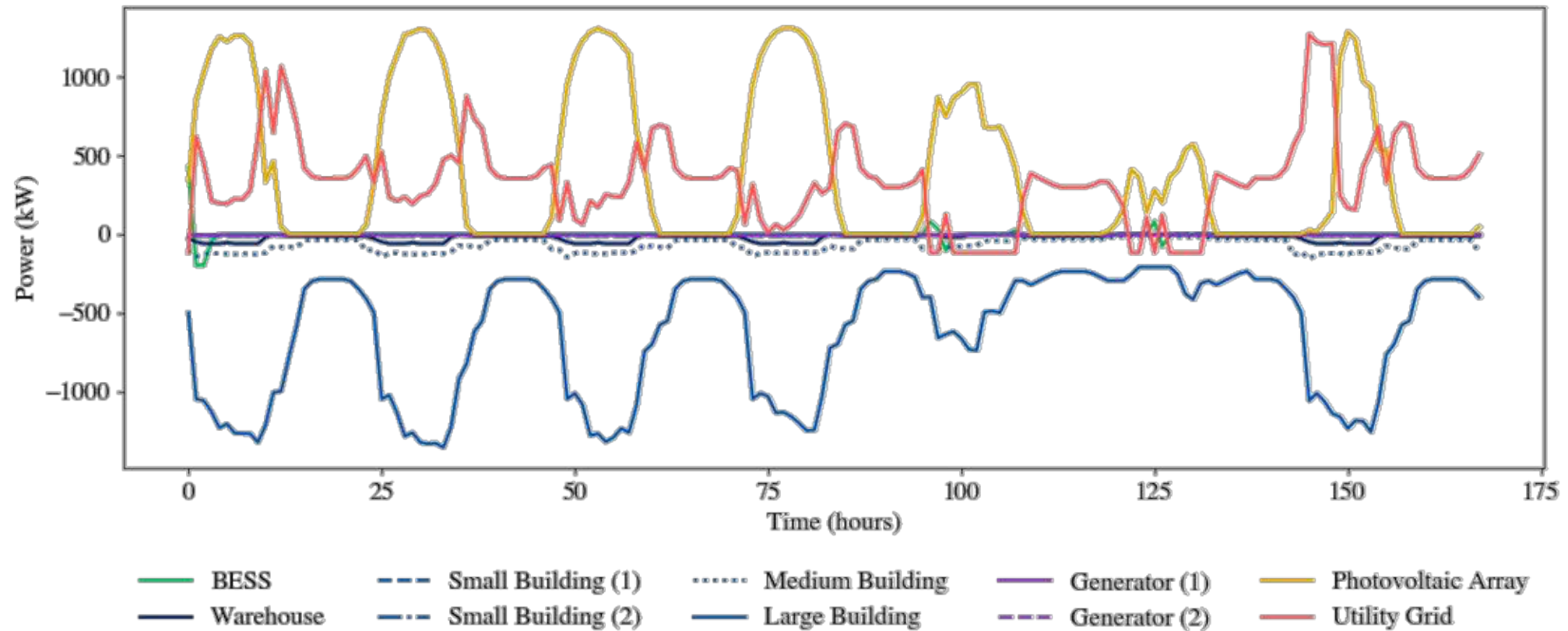
MG_LP_Test_v10.m ×
/Users/john_morris/Documents/Clemson/Research/MicrogridHg/archived_model/MG_LP_Test_v
423 %Initial Solve
424 x(:,n) = linsolve(A2,b);
425
426 %Determine if generator demand exceeds generat
427 Gen_Demand = -x(pos.Gens,n);
428
429 if any(Gen_Demand > [Gens.Capacity])
430     %If the B2 to B1 Buss line or BT1 line is
431     %exhausted cannot utilize ESS to make up g
432     if MG_State.B2_B1(n) == false || MG_State.
433         || BT1_Charge(n) < 0
434         % Below line was useful for debugging
435         % disp("Gens Overloaded at time step "
436         Overload.Gens = true;
437     else
438         %Otherwise set Generators at full outp
439         %make up unmet demand
440         A2(6,:) = pos.Gens & MG_State(n,:);
441         b(6) = -MG_State(n,{'B1_GEN1' 'B1_GEN2
442         x(:,n) = linsolve(A2,b);
443     end
444 %If Generator demand is negative, then generat
445 %charge batteries with excess PV generation
446 elseif any(Gen_Demand < 0)
447     A2(6,:) = pos.Gens;
448     b(6) = 0;
449     x(:,n) = linsolve(A2,b);
450 end
451
452 % Check for Battery Output Exceeded
453 % Add 0.1 Due to Rounding Errors in Linear Solver
454 if -x(contains(LoadVars,'BT1'),n) - 0.01 > BT1.Output * (BT1_Charge(n) > 0)
455     Overload.BT1 = true;
456     % Below line was useful for debugging in single runs. Commented out

```





Autonomously observes and simulates the real system

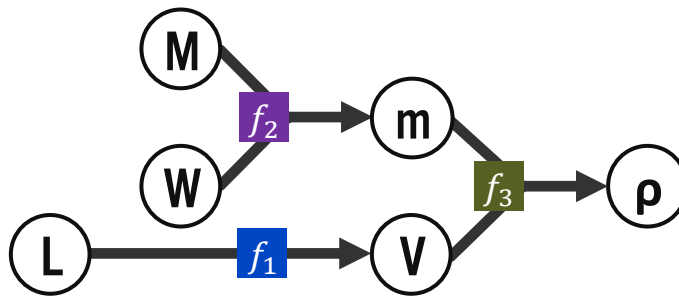


Digital twins require foundations that can handle modern challenges in modeling and simulation

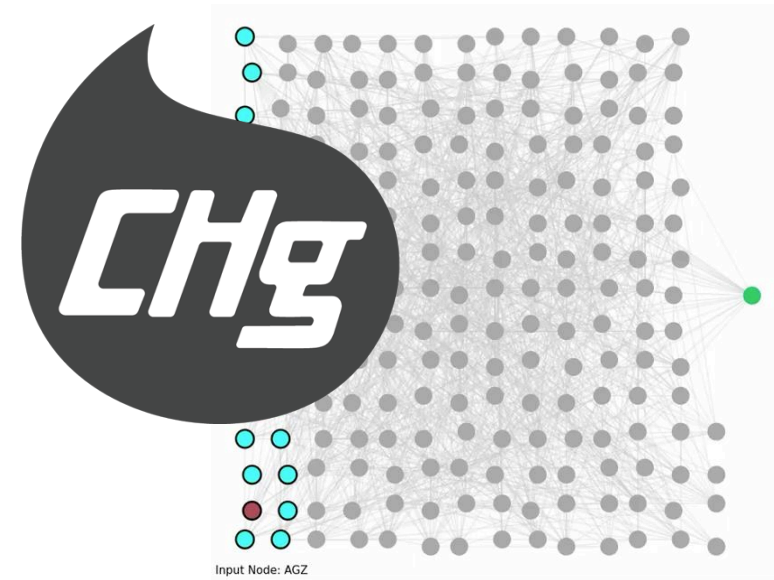
Digital twins are analogs of a real system



Twinned systems share the same CHG



Pathfinding leads to autonomous observation



John Morris
john.h.morris@uah.edu